What is array?

In array we can store n number of values of same type.

In array we can store primitive type values and ref type (objects) in array

1d - Primitive int array – creates one object int array object – 1 2 3 we can store the primitive values

1d String array - creates one string object – stores string objects

When single dim array?

To store list of values - primitive values / objects

When you create the array creates the array object .Inside that array object we can store primitive type values and objects.

When 2d / multi dim array?

To store table structure values - primitive values / objects

for loop and for each

int array

```
for (int i : intArray)
{   s.o.p(i)
}
```

Without using for loop how to print single dim array?

Arrays.toString(arrayVariable)

Without using for loop how to print 2 d array?

Arrays.deepToString(arrayVariable);

Arrays class – java.util

To use Arrays class – import java.util.Arrays

String

System

By default java imports the package java.lang to all the java programs.

1d – int array – count

```
int intArray[] = {10, 9, 8, 7};
```

intArray.length – count

2d

```
int arr[][] = {{1, 2, 3}, {2, 4, 5}, {4, 4, 5}};
```

```
arr.length = row count
arr[0].length = column count
arr[1].length = column count
arr[2].length = column count
```

Shallow copy and Deep copy(1d and 2d)

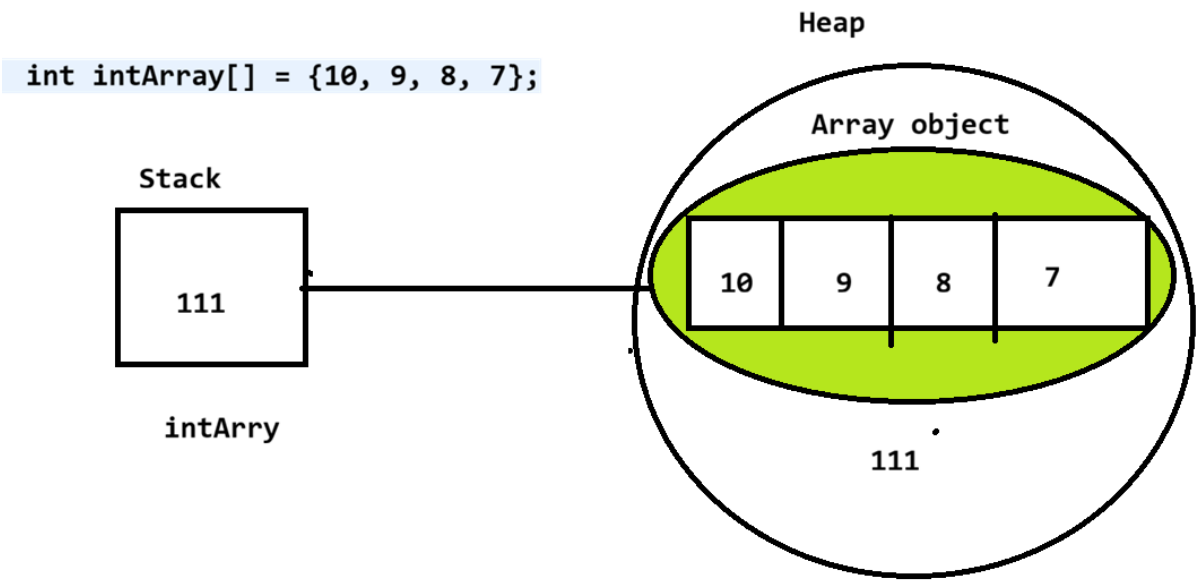
Shallow copy:

Making changes in one array affects another array

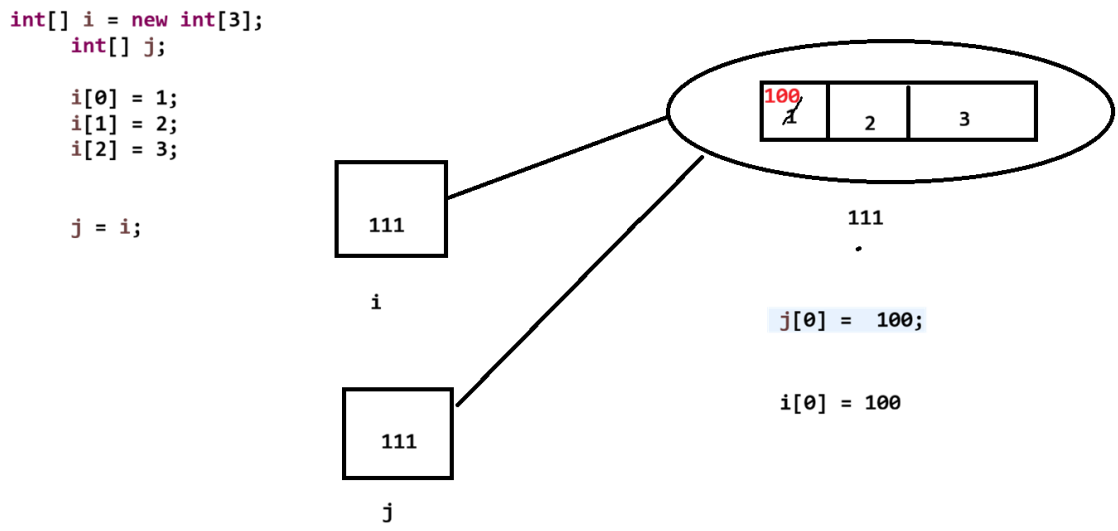
Deep copy

Making changes in one array will not affect another array

Single dim primitive int array

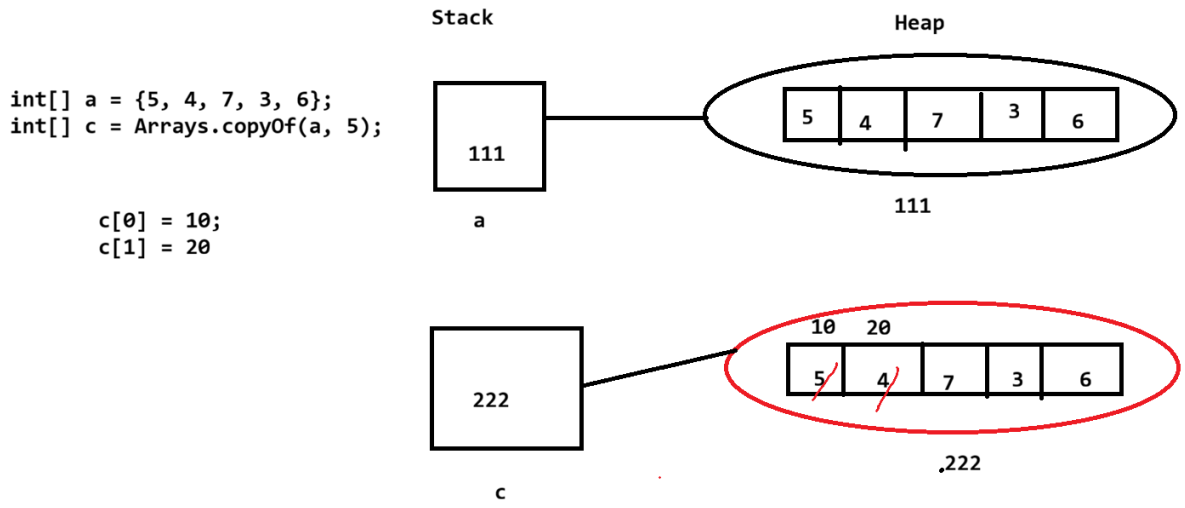


Assigning single dim int array to another array



Making change in j array affects I array -> shallow copy.

Using Arrays.copyof for single dim primitive int array



Arrays.copyof method creates the new array object (222)

That reference will be stored in c variable.

Single dim ref type array

```
int intArray[] = new int[5];
```

`new int[5];` creates the array object

```
int intArray3[] = {10, 9, 8, 7};
```

if you initialize the values directly then internally java creates the array.

```
int[] original = new int[]{10, 9, 8, 7};
```

```
int[] c = Arrays.copyOf(a, 5);
```

Arrays.copyOf method creates the array object with the size 5.

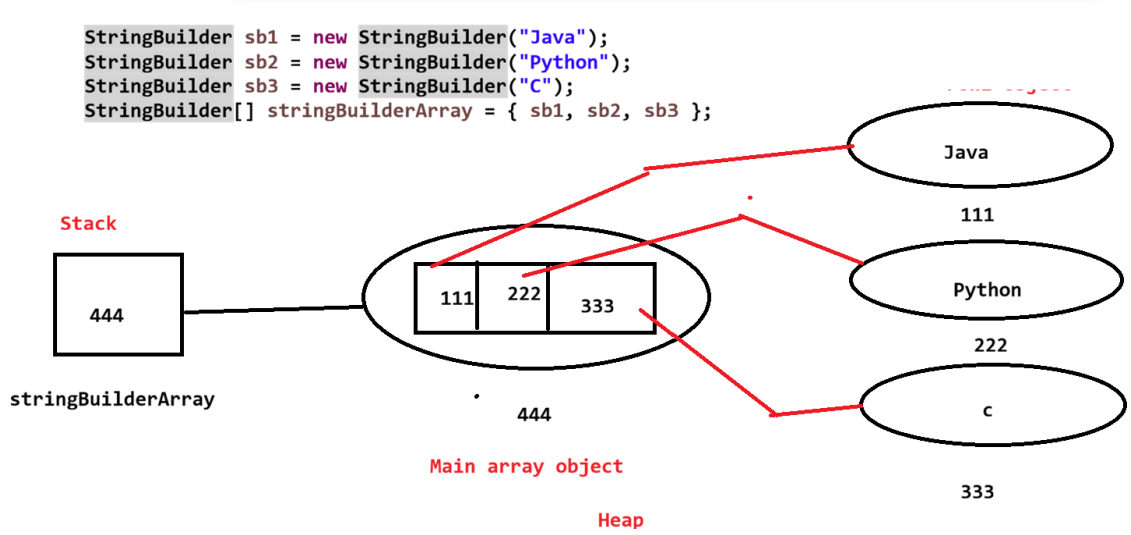
After using Arrays.copyof method , Making change in the c array will affect a array?

No, because c is holding different reference.

Making change in the c array is not affecting the a array – deep copy

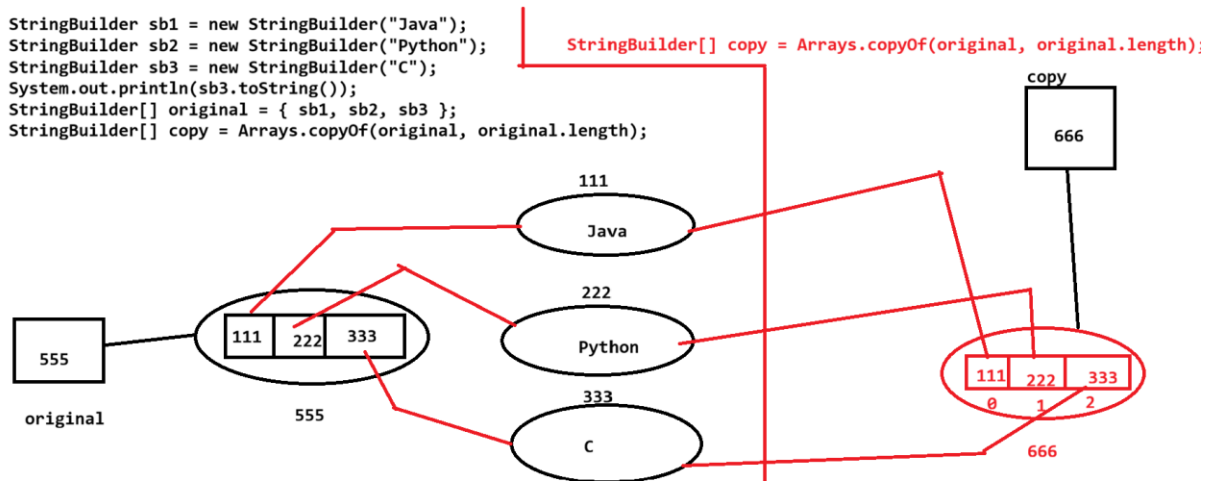
---

Array of string builder objects.



Stores the string builder object references in the main array object.

Single dim – ref type – shallow copy



Arrays.copyof method creates the array object(666).

Sicne original array contains the reference, reference is

Copied to the copy array. Two diff objects sharing the same

Refernces 111, 222, 333

Even though you use Arrays.copy of method to take the copy of array of string builder objects, results in shallow copy.

Becase here we have array of string builder objects.

Main array object holds the reference of string builder object.

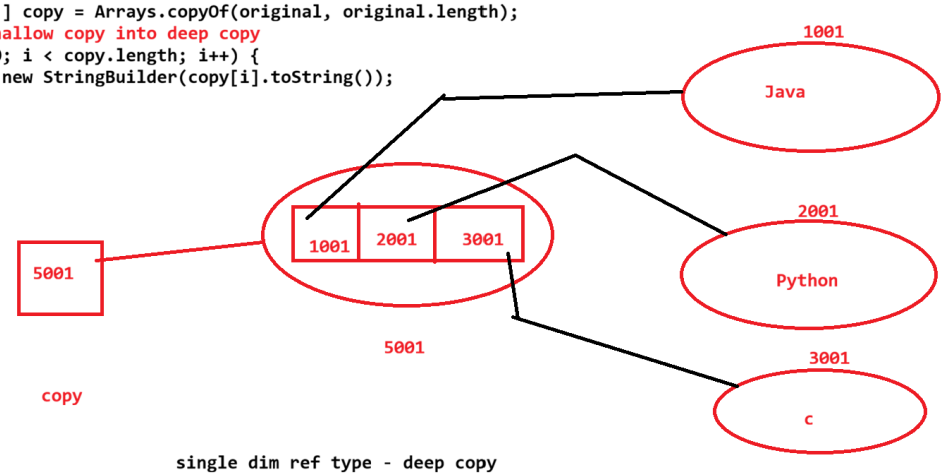
Copy method takes a copy of references.

Single dim ref type deep copy

```
StringBuilder[] copy = Arrays.copyOf(original, original.length);//4001

for (int i = 0; i < copy.length; i++) {
        copy[i] = new StringBuilder(copy[i].toString());
    }
```

```
StringBuilder[] original = { sb1, sb2, sb3 };
StringBuilder[] copy = Arrays.copyOf(original, original.length);
// Converts shallow copy into deep copy
for (int i = 0; i < copy.length; i++) {
    copy[i] = new StringBuilder(copy[i].toString());
}
```



single dim ref type - deep copy

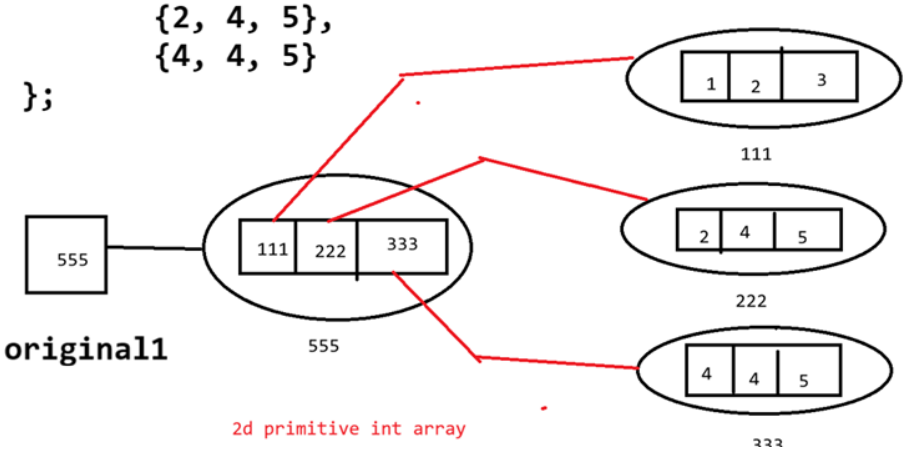
Single dim – primitive type array – Arrays.copyOf - deep copy

Single dim – ref type array – Arrays.copyof – shallow copy

So to get the deep copy we have to write the code.

2d int array – simple example

```
int[][] original1 = {
                {1, 2, 3},
                {2, 4, 5},
                {4, 4, 5}
        };
```

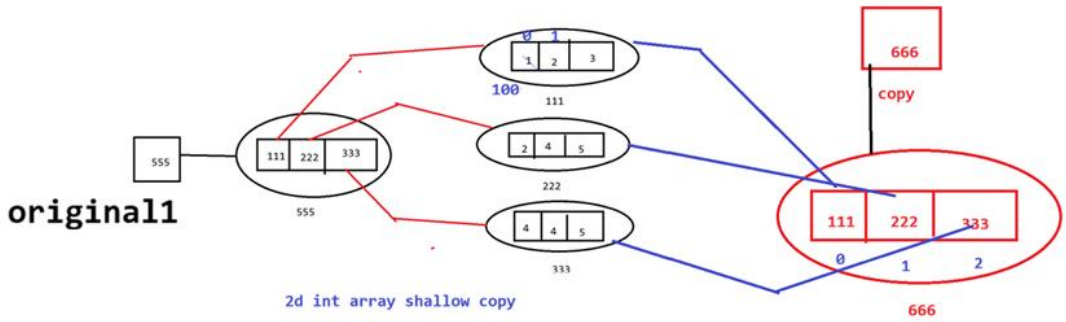


2d primitive int array

2d int array shallow copy

```
int[][] original1 = {
            {1, 2, 3},
            {2,4,5},
            {4,4,5}
        };

        int[][] copy = Arrays.copyOf(original1, original1.length);
```



Since the original array contains the reference, Arrays.copy of method takes the copy of references only. Because of this

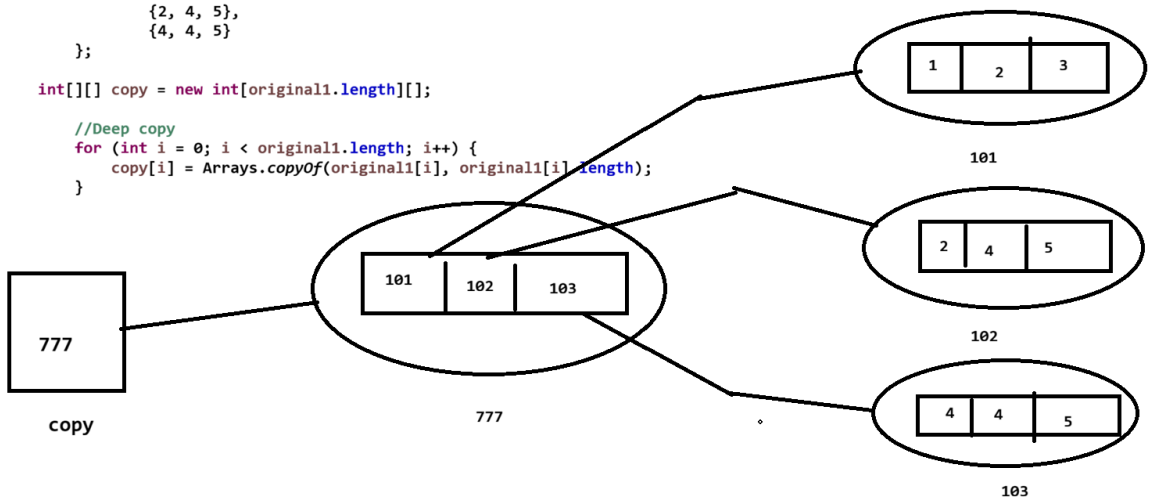
Two different objects is sharing the same references.

2d int array deep copy

```
int[][] original1 = {
                {1, 2, 3},
                 {2, 4, 5},
                 {4, 4, 5}
        };

    int[][] copy = new int[original1.length][];

        //Deep copy
        for (int i = 0; i < original1.length; i++) {
            copy[i] = Arrays.copyOf(original1[i], original1[i].length);
        }
```



1 d – primitive array – Arrays.copyof – deep copy

1d – ref type array – Arrays.copy of – shallow copy

2d – primitive int array – Arrays.copy of – shallow copy

2d – reference type – Arrays.copy of – shallow copy

From the shallow copy to get the deep copy , we have to write the code.

Ragged array / jagged array

In multidimensional array columns will be in different size in each row.

2 dim jagged array

```
int[][] myArray  = {
        {33, 71},
```

```
            {-16, 45, 50, -7},
            {99}
        };
myArray.length = row count.

myArray[0].length = gives the column count of first row.

        for (int i = 0; i < myArray.length; i++) //row
count
        {
            for (int j = 0; j < myArray[i].length; j++)
//column count in first row
            {
                System.out.print(myArray[i][j]);
                System.out.print("  ");
            }

            System.out.println();
        }
```

2 dim array

```
        int arr[][] = {{1, 2, 3}, {2, 4, 5}, {4, 4, 5}};
```

arr.length = row count

arr[0].length = column count

arr[1].length = column count

arr[2].length = column count

```
for (int i = 0; i < myArray.length; i++) //row count
        {
            for (int j = 0; j < myArray[0].length; j++)
//column count in first row
            {
                System.out.print(myArray[i][j]);
                System.out.print("  ");
```

```
            }

            System.out.println();
        }
```

2d ref type